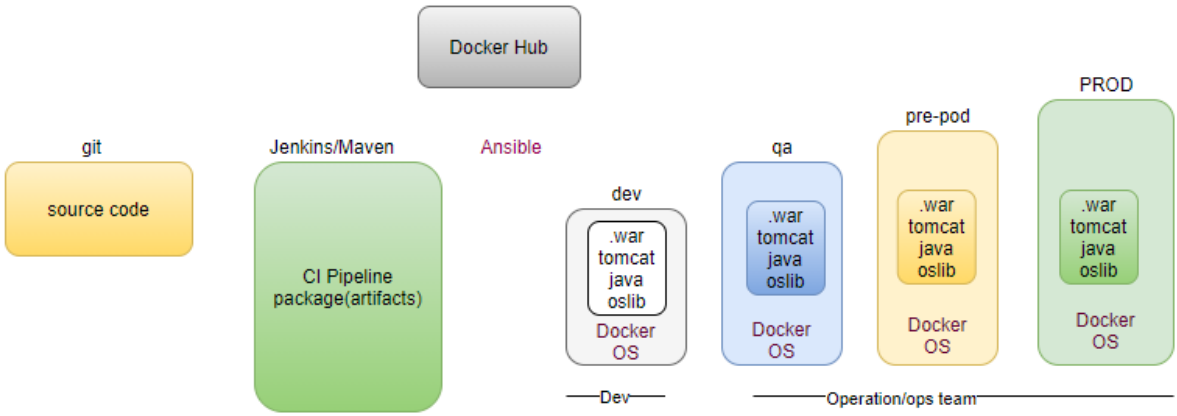
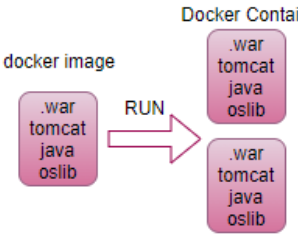
Day 6:

Docker:



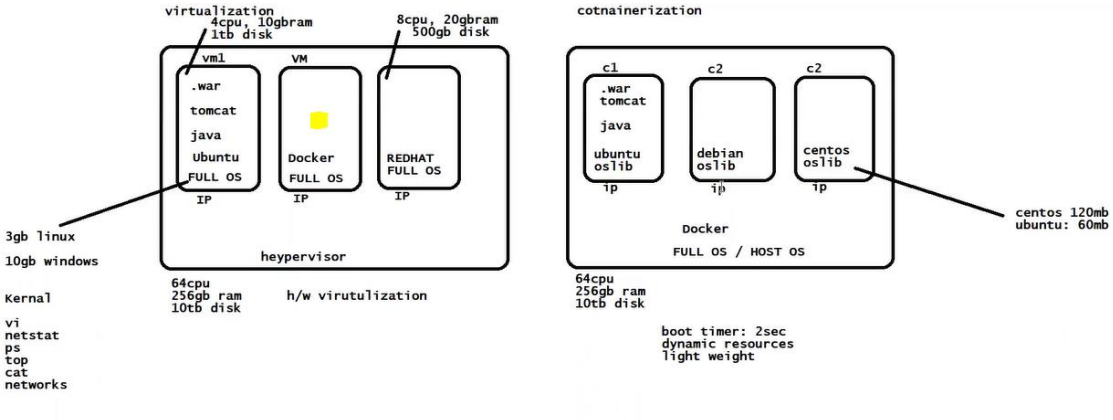
How to build the container and how to run the containef?
containerization tool:
- helps to build and run the container.
ex: docker,containerd,lxc,rocket

Docker-- containerization tool
BUILD-SHIP-RUN



container :
Is a run time instance of an image
Is a process that runs on your server

Difference between VM and Container?



step0: IAM role and programmatically access.

step1: Create EC2 Instance

step2: Install docker

docker info

step3: run two different containers on the docker (pull image from docker hub)

docker pull training/webapp

docker run -d -p 80:5000 training/webapp:latest python app.py

docker run --name some-httpd -d -p 8080:80 clearlinux/https

step3.1: Image exposes the port on host, must be enabled in SG (80,8080).

step3.2: docker ps # docker container ls

step4: create a Dockerfile corresponding to your app.

step5: docker build -t demo-001 .

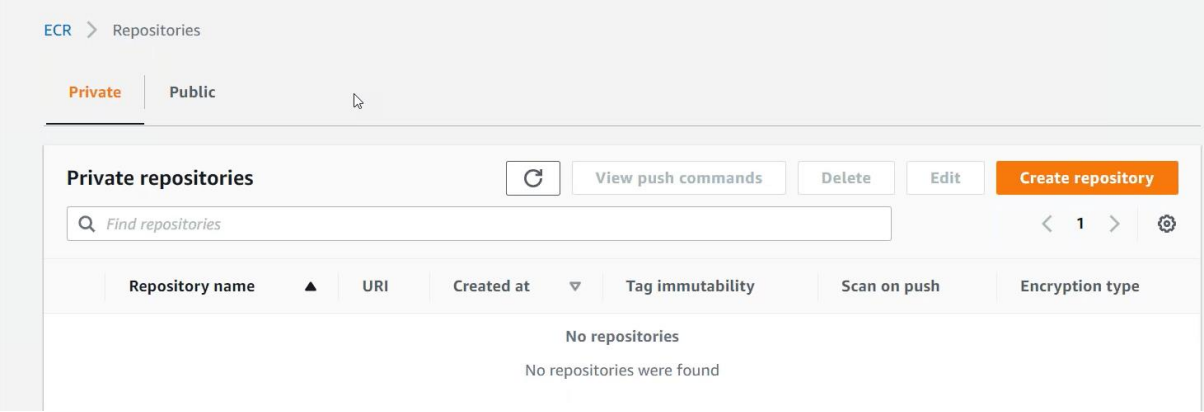
step5.1: docker run -d -p 80:80 demo-001:latest

step5.2: validate in browser <IP>

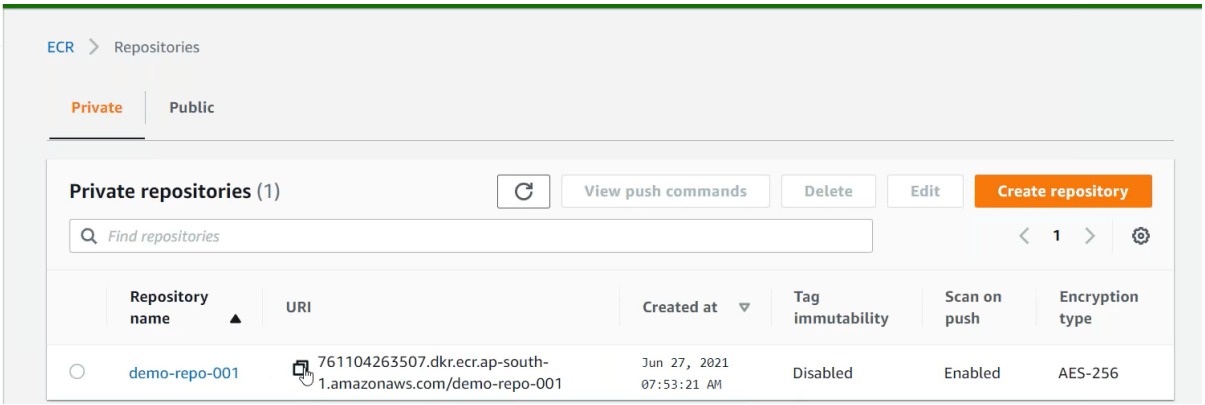
ECR:

Now can create public repository just like dockerhub

step6: create Repository in AWS ECR (Elastic Container Registry)



aws ecr create-repository --repository-name demo-repo-001 --image-scanning-configuration **scanOnPush**=true --region <region-code> # vulnerability



step7: tag (alias) the local image with AWS ECR URI

docker tag <image-name/demo-001>:latest <your aws account ID>.dkr.ecr.<your region code>.amazonaws.com/demo-repo-001:latest

step8: get password (will not show, stores locally)

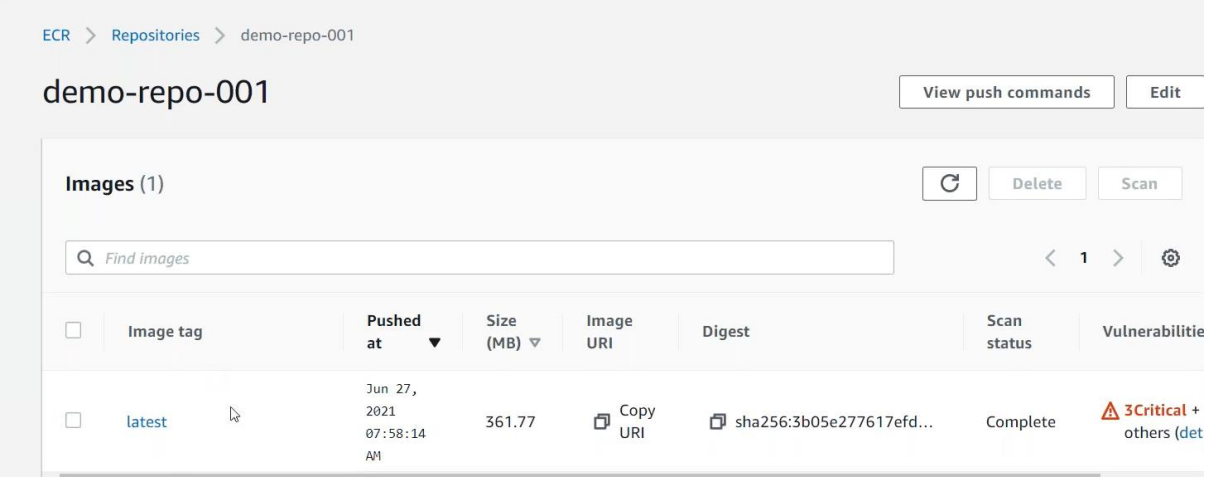
aws ecr get-login-password --region <your region code> | docker login --username AWS --password-stdin <your aws account ID>.dkr.ecr.<your

region code>.amazonaws.com

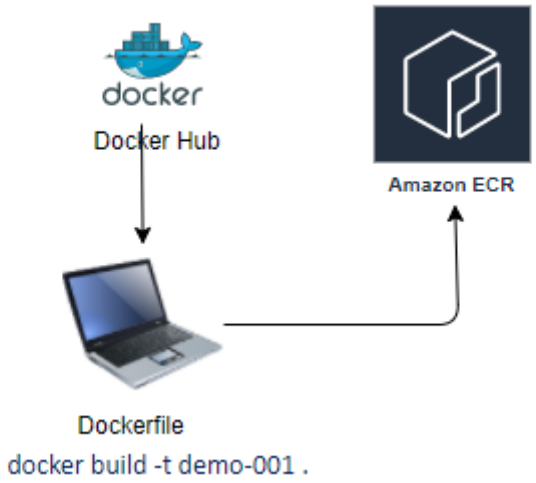
step9: push the repository in AWS ECR

docker push <your aws account ID>.dkr.ecr.<your region code>.amazonaws.com/demo-repo-001:latest

```
aws ecr create-repository --repository-name demo-repo-001 --image-scanning-configuration scanOnPush=true --region ap-south-1

docker tag demo-001:latest 761104263507.dkr.ecr.ap-south-1.amazonaws.com/demo-repo-001:latest

aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 761104263507.dkr.ecr.ap-south-1

docker push 761104263507.dkr.ecr.ap-south-1.amazonaws.com/demo-repo-001:latest


761104263507.dkr.ecr.ap-south-1.amazonaws.com/demo-repo-001
```



We have an image in ECR



ECS:

Amazon Elastic Container Service (Amazon ECS) is a highly scalable, fast container management service that makes it easy to run, stop, and manage containers on a cluster. **Your containers are defined in a task definition that you use to run individual tasks or tasks within a service.** In this context, a **service is a configuration that enables you to run and maintain a specified number of tasks simultaneously in a cluster.** You can run your tasks and services on a serverless infrastructure that is managed by AWS Fargate. Alternatively, for more control over your infrastructure, you can run your tasks and services on a cluster of Amazon EC2 instances that you manage.

step1: Create a task definition (pod).

Launch Types:

- FARGATE (serverless aws managed)
  - price based on task size.
  - Requires network mode awsvpc
  - AWS-managed infrastructure.
- EC2
  - Price based on resource usage.
  - multiple network modes available.
  - Self-managed infrastructure using Amazon EC2 instances.
- EXTERNAL
  - price based on instance-hours and additional charges for other AWS services used.

- Self-managed on-premises infrastructure with ECS anywhere

A task definition specifies which containers are included in your task and how they interact to each other, you can also specify data volumes for your containers to use.

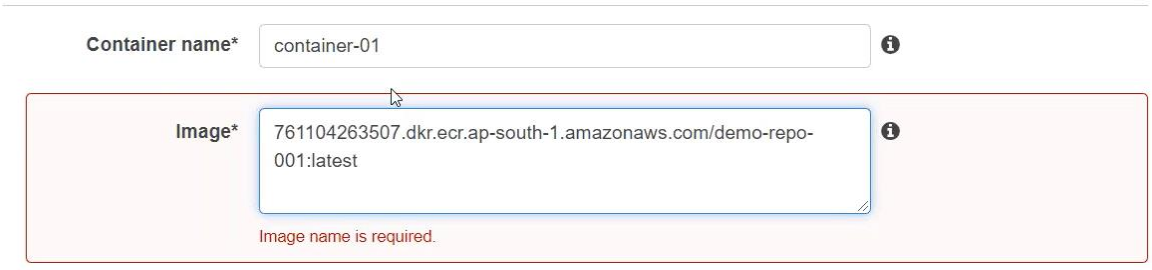
A task definition is required to run Docker containers in Amazon ECS. The following are some of the parameters you can specify in a task definition:

- The Docker image to use with each container in your task
- How much CPU and memory to use with each task or each container within a task
- The launch type to use, which determines the infrastructure on which your tasks are hosted
- The Docker networking mode to use for the containers in your task
- The logging configuration to use for your tasks
- Whether the task should continue to run if the container finishes or fails
- The command the container should run when it is started
- Any data volumes that should be used with the containers in the task
- The IAM role that your tasks should use

You can define multiple containers in a task definition. The parameters that you use depend on the launch type you choose for the task.

Your entire application stack does not need to be on a single task definition, and in most cases it should not. Your application can span multiple task definitions. You can do this by combining related containers into their own task definitions, each representing a single component.

step1.1: Task need a container.

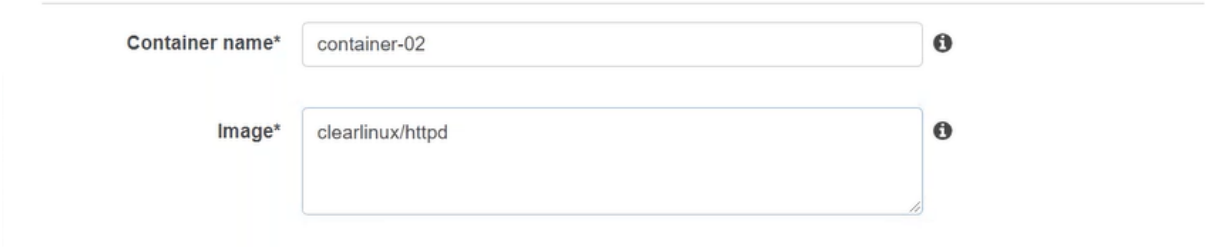


First will check the image from ECR, if image not available, will pull the image from DockerHub.com

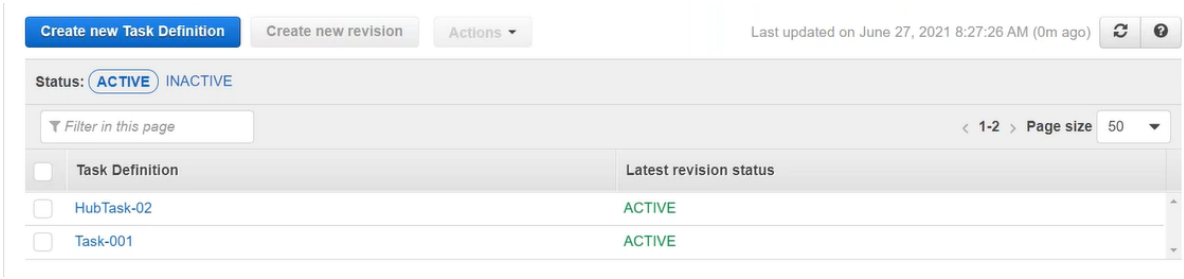
step2: Fargate (serverless for the container) is serverless even though we have to define the container Port

It specifies task which we are creating run on this type of container.

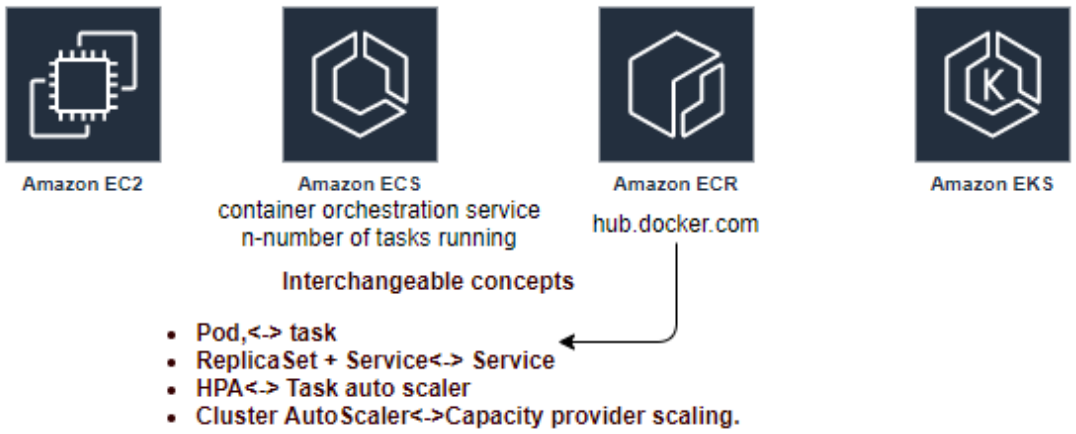
step3: Create a new task definition, get image from docker hub.



step3.1: compatible type: EC2

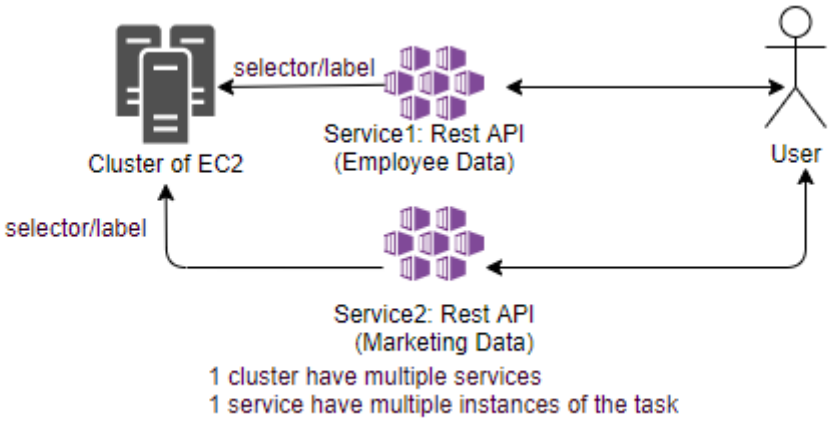


We have created two tasks (Fargate/EC2).



Cluster: An Amazon ECS cluster is a regional grouping of more container instances on which you can run tasks requests. Each account receives a default cluster the first time you use the ECS service, Cluster may contain more than one Amazon EC2 instance types. First thing is cluster all containers run inside the cluster.

step4: Create a cluster -networking- Fargate (Internally creates CloudFormation stack).



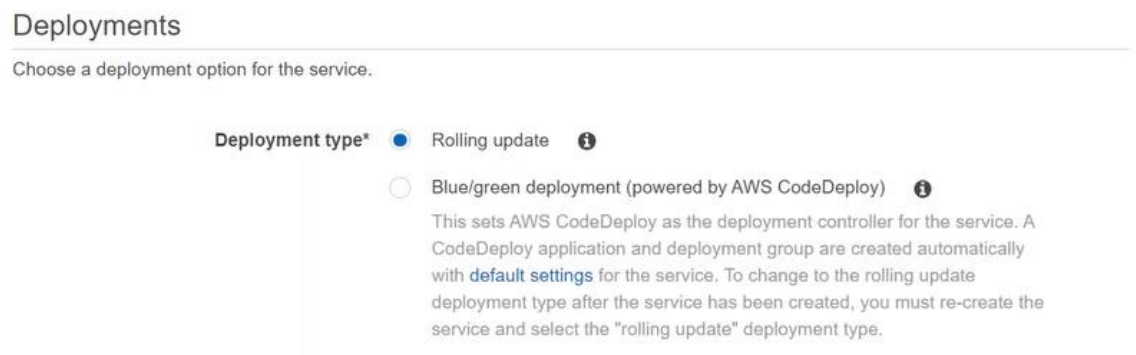
step4.1: Select the service tab, create a service (service can hold one task at a time, but holds multiple Instances of the tasks).

- Launch Type: Fargate
- Task Definition: Task-001

step4.2: get the public IP of the created tasks, and validate from browser.

step4.3: Select the service tab, create a service (service can hold one task at a time, but holds multiple Instances of the tasks).

- Launch Type: EC2
- Task Definition: hub-task
- Service type: Replicas/ Daemon

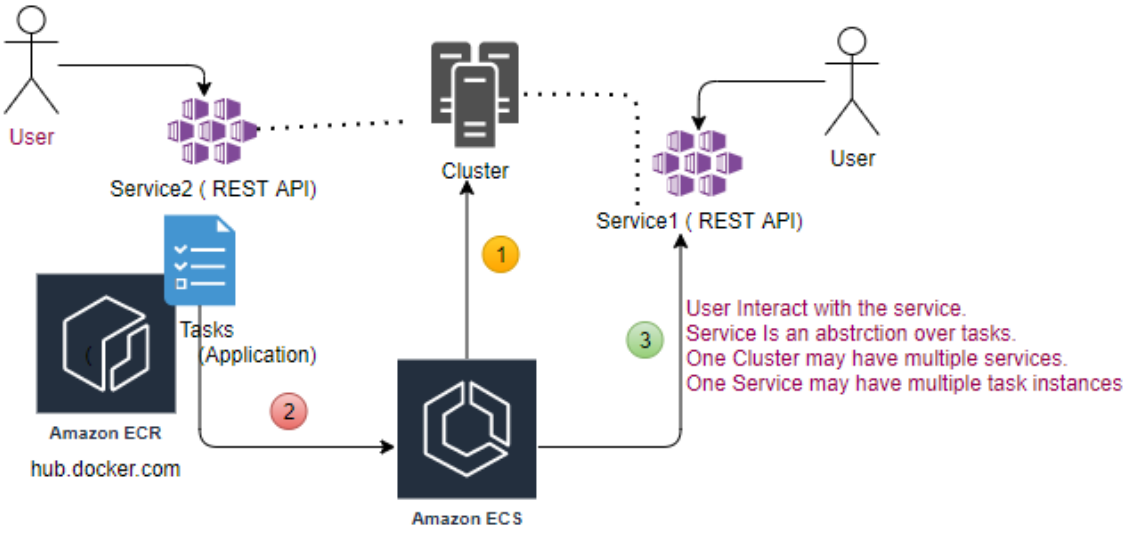


CloudWatch Container Insight: Is a monitoring and troubleshooting solutions for containerized application and microservices, it collects, aggregates and

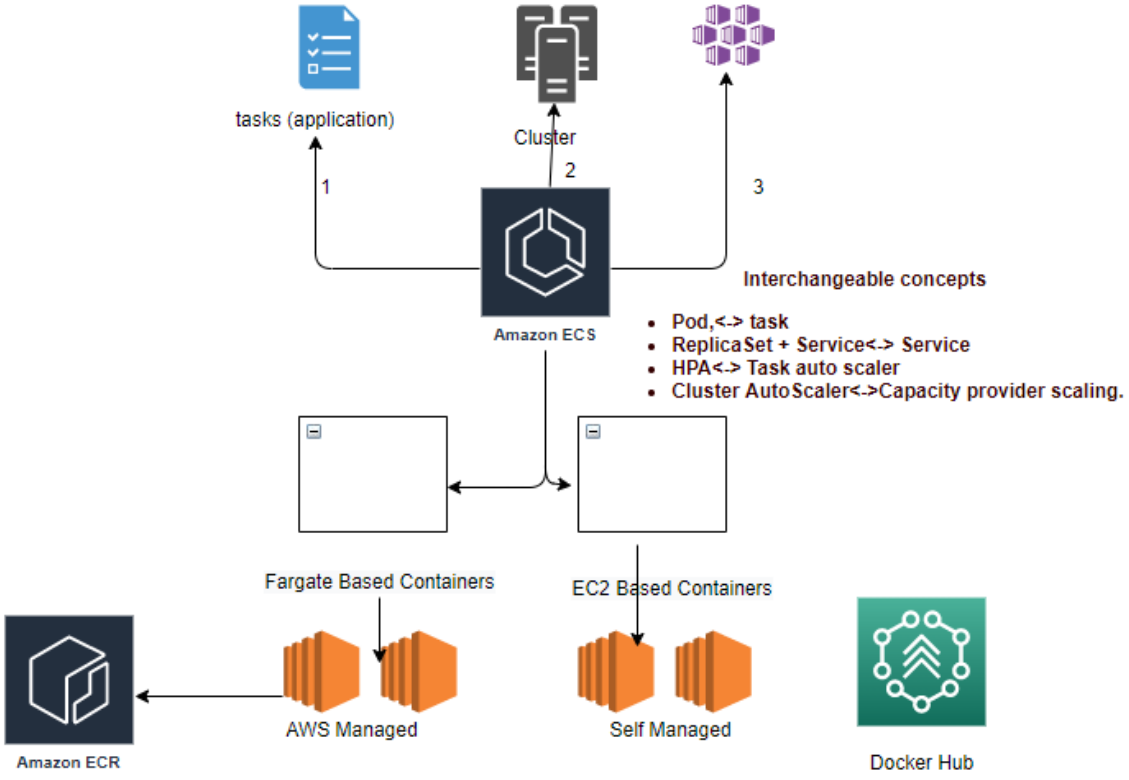
summarize compute utilization such as CPU, memory, disk and network, and diagnostic information such as container restart failure to help you isolate issues with your cluster and resolve them quickly.

step4.4: get the public IP of the created tasks, and validate from browser.

step4.5: ECS (group of EC2), creates EC2 instances (managed by AWS).



Tasks are deployed by ECS service on the EC2, not done manually.



Task (Pod contains image)

Who is managing server (if managed by aws then for us it's serverless)?

step4.6: Change the version of app.py , and follow the same steps.

step6: change the code, push the image in ECR.

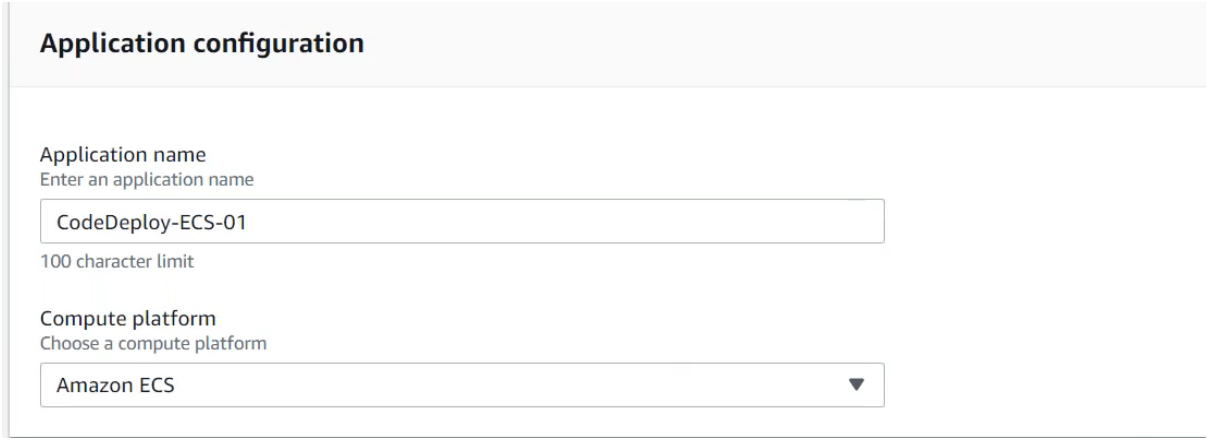
step7: Latest version will be created in ECR.

step7.1: create a new task version using the latest version.

How to bring into new version?

Using the Code Deploy

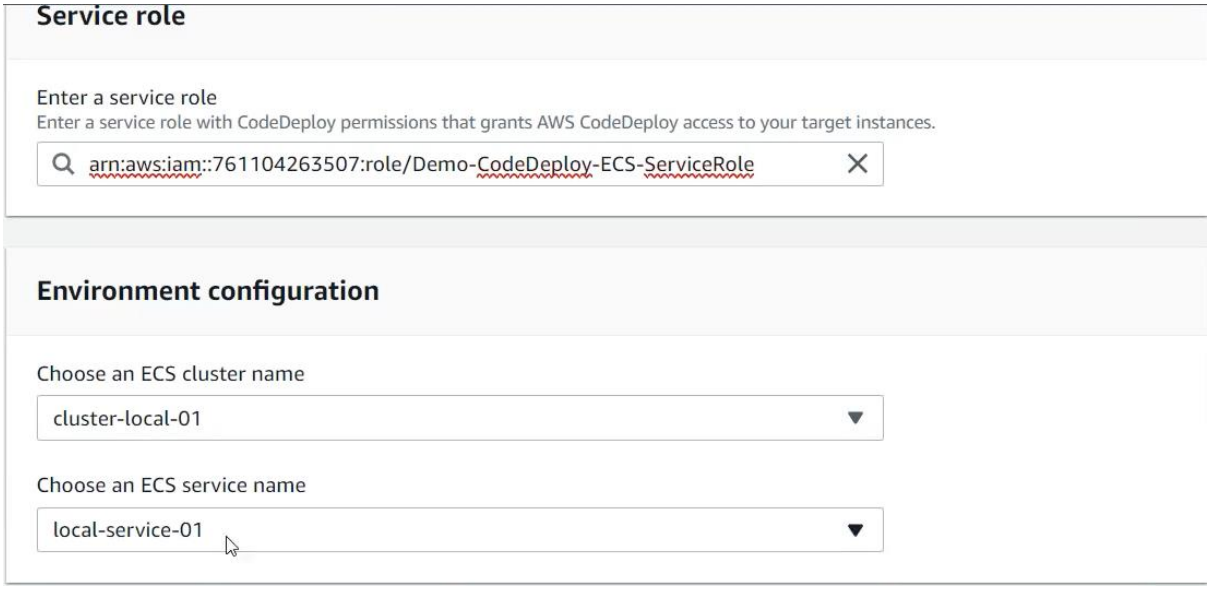
step8: Go to CodeDeploy=> Create application



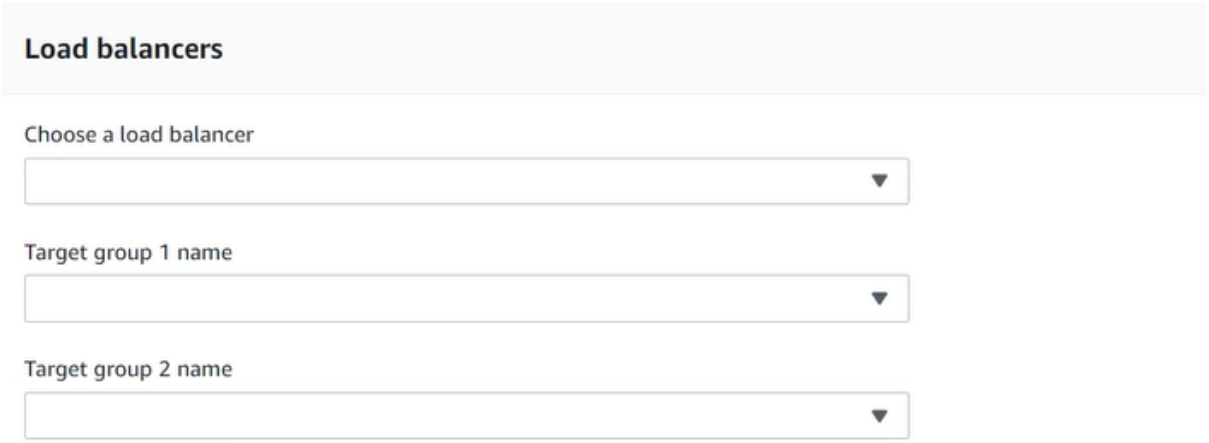
step8.1: Create Deployment Group

step8.2: IAM role **Demo-CodeDeploy-ECS-Service Role**

AWSCodeDeploy-ECS-ServiceRole



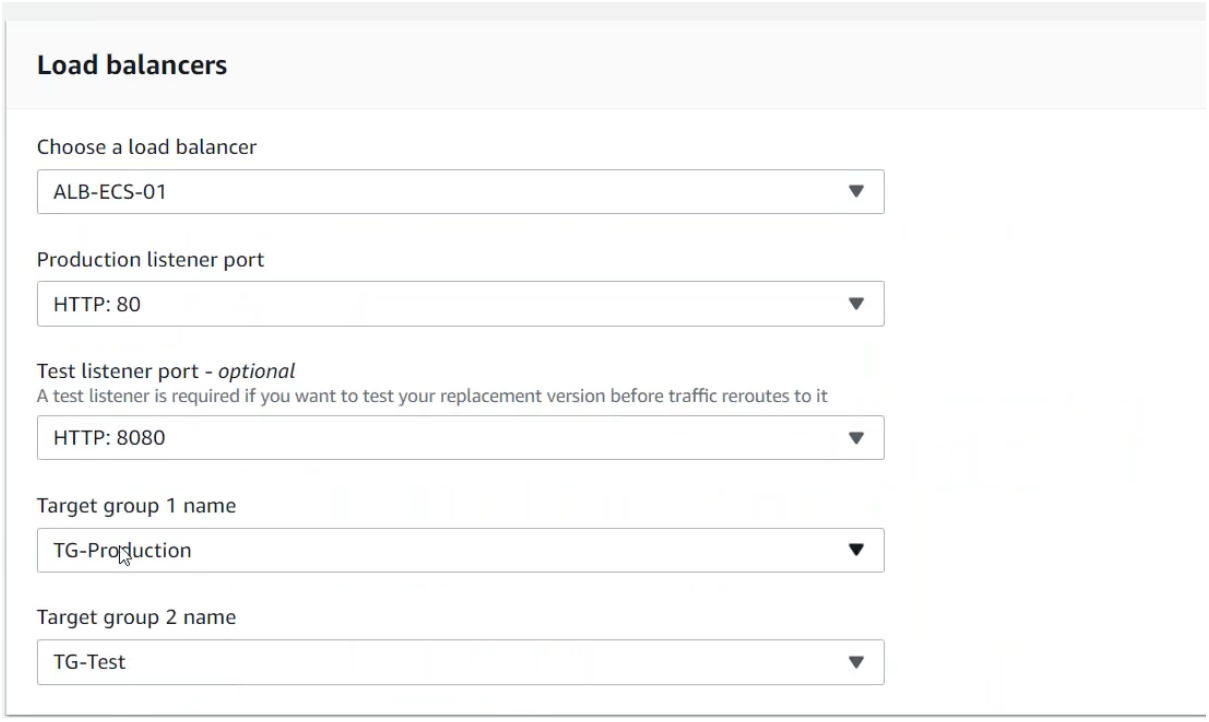
All container-based deployments are mandatory to run through LB.



step8.3: create ALB

step8.3.1: create two Target Groups TG-production, TG-test

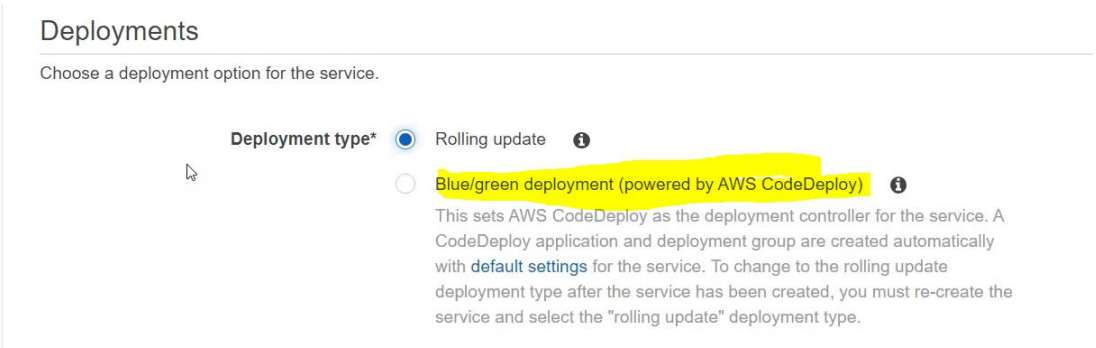
step8.4: Update Code Deployment Group



Deployment Groups ECS service must be configured for a CODE_DEPLOY **deployment controller**. (Rolling Update not managed by CodeDeploy).

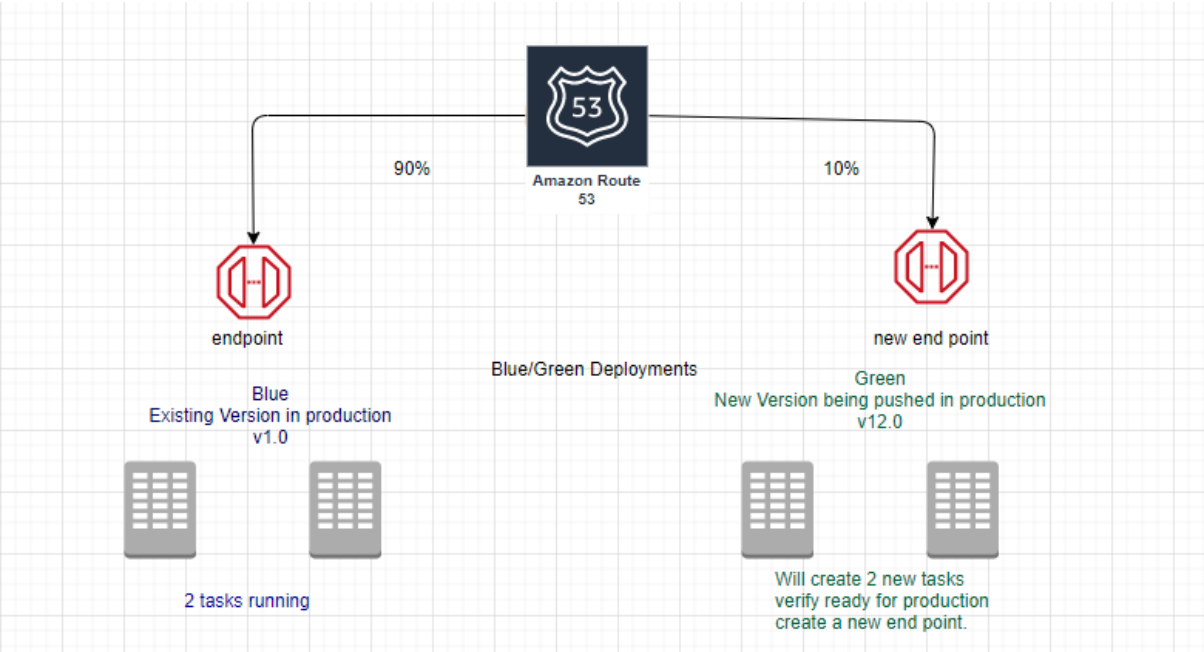
Update the service. Only option Blue/green available for CodeDeploy through the ECS.

Rolling update must be done by our self manually.



Blue: Existing Version in Production

Green: New version being pushed in Production



Will see 4 tasks, 2 with older version and 2 with new version.

step8.5: Deployment Configurations

CodeDeployDefault.OneAtATime

Compute platform
EC2/On-premises
Minimum healthy hosts value
1

CodeDeployDefault.HalfAtATime

Compute platform
EC2/On-premises
Minimum healthy hosts value
50%

CodeDeployDefault.AllAtOnce

Compute platform
EC2/On-premises
Minimum healthy hosts value
0

CodeDeployDefault.LambdaAllAtOnce

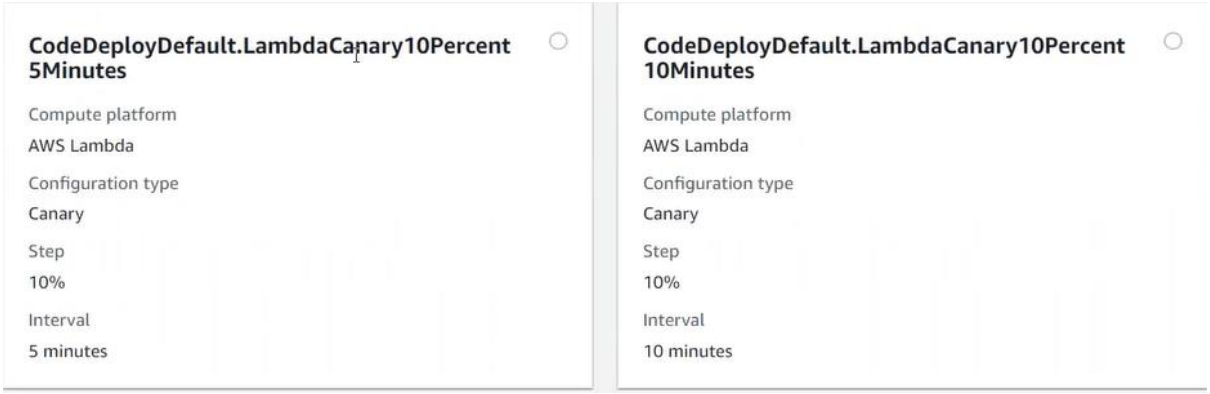
Compute platform
AWS Lambda
Configuration type
All at once

CodeDeployDefault.LambdaLinear10PercentEvery1Minute

Compute platform
AWS Lambda
Configuration type
Linear
Step
10%
Interval
1 minutes

CodeDeployDefault.LambdaLinear10PercentEvery2Minutes

Compute platform
AWS Lambda
Configuration type
Linear
Step
10%
Interval
2 minutes



step8.6: create a new service v2.0

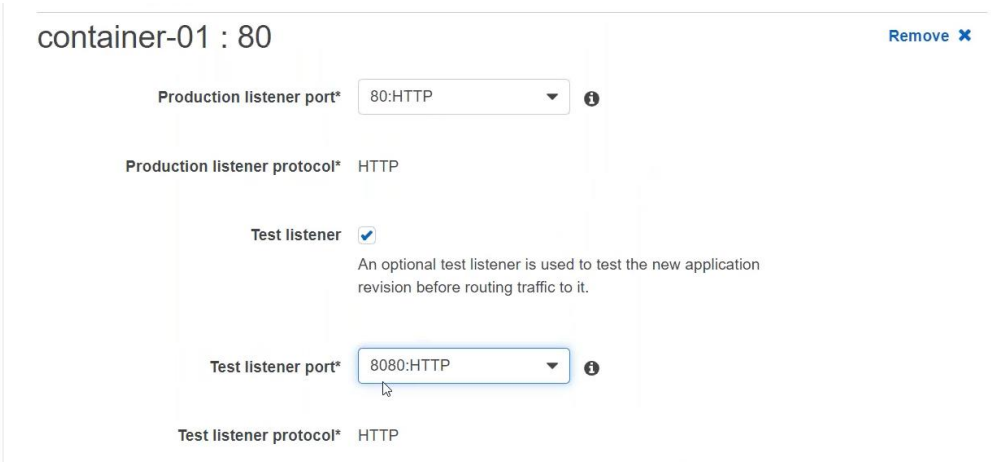
Launch Type: Fargate

Service Type: Replica

Deployments: Blue/green

Add Application Load balancer

Container: Add to the Load balancer



step8.7: Target Group and CodeDeploy Application created by ECS service.

If you create an ECS service using blue-green deployment, this selection of blue/green type itself create a CodeDeploy application, also creates additional rules (listeners) with in ALB you specify. It also creates Deployment Group Detail.

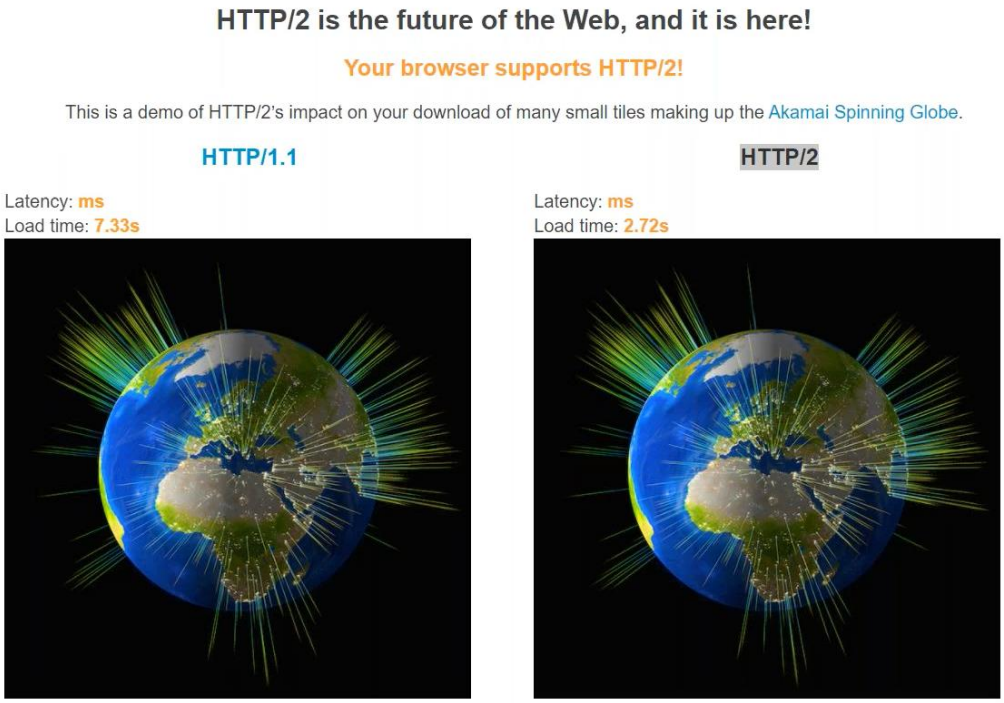
step8.8: Create deployment

Difference between linear (step by step) and canary deployment (two steps)?

gRPC vs REST?

"gRPC is roughly 7 times faster than REST when receiving data & roughly 10 times faster than REST when sending data for this specific payload. This is mainly due to the tight packing of the Protocol Buffers and the use of HTTP/2 by gRPC."

Http2 demo



AWS Kubernetes Workshop

Ref:

https://drive.google.com/file/d/1BF_rO7DigC6SbZ2v5Gqwv3E59RJGeftt/view?usp=sharing